A FRIENDLY, PRACTICAL GUIDE

DAN KK Recycling Centre System

What the system is for, how to use it, how the code works, and what changes when code or records are removed.

> **In one sentence:** this is an offline-capable, browser-based local application for recording recyclable-material deliveries, keeping supplier history, and reviewing the centre's totals.

Prepared from the project source code

Plain-language walkthrough for staff, developers, and colleagues

1. Purpose and big picture

The system helps DAN KK Recycling Centre keep a consistent record of material received at the centre. For each delivery, a staff member records who brought it, what kind of material it was, its weight, and the agreed rate per kilogram. The system calculates an estimated value, saves the intake on this computer, and updates the dashboard and supplier history.

Why it exists

- **Traceability:** each saved intake has a date/time, supplier or collector, material, weight, rate, and value.
- **Visibility:** staff can see total material weight, intake count, estimated value, recent records, and supplier contributions.
- **Continuity:** saved records live in a local SQLite database and remain after the app closes or the computer restarts.
- **Simplicity:** users work in a web browser; they do not need to use a command-line database tool.

Staff browser → local Java web server → SQLite database → results sent back to browser

The Java process serves the web pages and handles requests. It listens only on this computer at `127.0.0.1:8765`. The interface is contained in the Java server source, not a separate web application folder. The app needs no external service for normal operation.

> **Important scope:** this is a single-computer local app, not an internet-hosted or shared multi-computer service. “Local-only” does not mean the database is encrypted or that the app has user accounts. The inspected interface has no login or role-management feature.

The five visible pages

Page	What it is for
Home	Centre introduction, shortcuts, and live totals.
About us	Explains the centre's purpose and values.
Services	Describes material intake and supplier services; links to the working pages.
Dashboard	Records a material intake and displays all-time totals and the latest 100 intakes.
Suppliers	Adds, searches, renames, and (where allowed) deletes supplier/collector names; shows their contribution totals.

2. How to start and use it

Start the application

1. Make sure Java 25 is installed. The project is configured for Java 25.
2. On Windows, double-click `run.bat` in the project folder, or run `.\run.bat` from PowerShell in that folder.
3. Keep the command window open. It runs the Java server; closing it stops the app.
4. The app tries to open the browser at `http://127.0.0.1:8765/`. If it does not, enter that address manually.

The launcher checks Java 25 and the locally cached SQLite JDBC library, recompiles the Java source, then starts `ug.dankk.App`. If the dependency is not cached, the README says to connect once and run `mvn dependency:go-offline` from the project folder. After the required library is cached, ordinary operation is local/offline.

Record an intake

1. Open **Dashboard**.
2. Enter the supplier/collector name (you can choose a suggested existing name).
3. Select one of: Plastic, Paper / Cardboard, Glass, Metal, E-waste, or Other.
4. Enter a positive weight in kilograms, up to 3 decimal places.
5. Enter a positive rate per kilogram, up to 2 decimal places.
6. Select **Save intake record**. If accepted, the form clears and the totals/recent list refresh.

Example: 2.500 kg at 1.20 per kg gives 3.00 in estimated value. The application formats money using the computer's currency/locale settings; it does not define a separate fixed currency in the code.

Manage a supplier

- **Add:** enter a name on Suppliers and add it. The intake form also creates a supplier entry automatically when an intake is saved under a new name.
- **Search:** type part of a name to filter the displayed list.
- **Rename:** the new name is applied both to the supplier list and to matching historical intakes, so the history stays connected.
- **Delete:** only a supplier with no intake history can be deleted. Names with recorded intakes are protected so their records are not orphaned.

> **Stop the app:** press Ctrl+C in the launcher window. A shutdown hook asks the server to stop. Stopping the server does not delete the database.

3. What happens behind the scenes

Think of a saved intake as a message travelling through four steps:

1. **Browser form:** the page checks required fields and sends the values to `POST /api/intakes`.
2. **Java HTTP server:** it accepts the local request, reads the form, checks the supplier name, material category, positive numbers, decimal precision, and request size.
3. **Database layer:** Java calculates/stores the values and inserts the intake into SQLite. A new supplier name is added if needed.
4. **Refresh:** the server replies that it was saved. The browser requests `/api/dashboard`, receives the latest totals and recent entries, and redraws the page.

Fill in form → validate → calculate → save in SQLite → refresh totals and history

Other browser/server requests

Request	Purpose
`GET /`	Returns the page (HTML, CSS, and JavaScript are embedded in the Java source).
`GET /api/dashboard`	Returns all-time transaction count, total weight, total value, and up to 100 newest records.
`GET /api/suppliers`	Returns suppliers and each one's transaction count, total weight/value, and last intake time.
`POST /api/suppliers`	Adds a supplier name.
`PUT /api/suppliers`	Renames a supplier and updates its historical transaction names together.
`DELETE /api/suppliers`	Deletes a supplier only when it has no intakes.

How the money calculation works

Weight is accepted to 0.001 kg; rate is accepted to 0.01 per kg. The code uses decimal arithmetic (`BigDecimal`) for the calculation. It multiplies weight by rate, rounds the resulting amount to two decimal places using half-up rounding, then stores money as integer cents. For example, 1.250 kg × 2.40/kg = 3.00, stored as 300 cents. This avoids common binary floating-point errors in currency calculations.

Validation and error feedback

The browser provides basic required/minimum/step checks, but Java validates again because requests should not be trusted just because they came from the form. Invalid values get a clear client error; duplicate supplier names are rejected; unexpected database failures are logged in the server console and reported as a server error. The server limits form request bodies to 8,192 bytes and uses prepared SQL statements for values.

4. What is saved, and what is not

The database file is `data/dankk-recycling.db`, relative to the application working/project directory. On startup, the database code creates the `data` folder if needed and creates the tables if they do not exist.

Table	Saved information
`transactions`	Each intake: generated ID, supplier name, material, weight, rate in cents, total amount in cents, and timestamp. Positive weight, rate, and value are enforced by database checks.
`suppliers`	Supplier name and creation timestamp. Names are unique without regard to letter case. The app also imports names from older transaction data during initialization.

- Dashboard totals are calculated from all stored transactions.
- The dashboard shows only the most recent 100 individual intakes; older intakes remain saved and counted in all-time totals.
- The supplier page includes suppliers even when they have no intakes, with zero totals.
- Intakes are not deleted from the interface. Supplier deletion is blocked when historical transactions exist.
- There is no cloud synchronization, external API, automatic off-site backup, built-in export, or user login in the inspected source.

> **Protect the records:** the SQLite file contains the system's business history. Back it up before changing code or moving the project. For a simple safe copy, stop the app first, copy `data/dankk-recycling.db` to a safe approved backup location, then restart. Do not delete it to “clean up code.”

Local security measures visible in the code

The server binds specifically to `127.0.0.1` rather than all network interfaces. It checks the expected local Host/Origin, limits request size, uses prepared statements, emits browser security headers, and disallows external connections in its Content Security Policy. These are useful protections for a local app; they do not replace account access controls, OS permissions, backups, or database encryption.

5. Project files: what each one does

File/folder	Role in the system	If it is removed
`src/main/java/ug/dankk/App.java`	Java entry point. Initializes the database, starts the HTTP server, opens the browser, waits while running, and registers shutdown cleanup.	The configured app has no `ug.dankk.App` entry point; the launcher/build cannot start the system.
`src/main/java/ug/dankk/RecyclingHttpServer.java`	Runs the local web server; contains the complete page markup, styles and browser JavaScript; routes requests; validates forms; produces JSON responses.	The browser interface and HTTP/API behavior disappear. `App` also cannot compile while referring to the missing server class.
`src/main/java/ug/dankk/RecyclingDatabase.java`	Creates/connects to SQLite; defines the tables and database operations; records intakes, manages suppliers, and calculates summary/history data.	The app cannot compile/run its database features. Existing data file is not automatically deleted, but there is no working code to read/write it.
`pom.xml`	Maven project identity, Java 25 compiler level, SQLite JDBC dependency, and configured main class.	Maven no longer has the project's declared build/dependency setup. The batch launcher uses a local jar path directly, but this is not a reason to remove the project manifest.
`run.bat`	Windows launcher: checks Java/Javac 25 and cached SQLite driver, compiles source into `target/classes`, and starts the app.	Double-click/PowerShell startup convenience is lost. The program can still be started manually only if compiled and its runtime classpath are set up correctly.
`README.md`	Human instructions: requirements, launch steps, features, validations, and database location.	The application may still run, but users lose the project's written setup and usage instructions.

`data/dankk-recycling.db`	The actual saved supplier and intake data.	Business records are lost if the file is permanently deleted. On next run, the app creates a fresh empty database and tables.
`target/`	Generated build output (compiled classes and packaged artifacts).	Generated output can normally be rebuilt from source; the launcher itself deletes/recreates `target/classes`. It is not the authoritative record store.
Root `Loginframe.java`	An empty Java interface; it has no methods or login logic and is not part of the current launcher flow.	No current system behavior should change; there is no implemented login function to remove.
`src/main/java/ug/dankk/MainFrame.java`	Present but empty (zero bytes) in the inspected workspace.	No current behavior should change; the application is browser-based and does not use a Java Swing main window.

> **Useful distinction:** Java source code controls what the program does. The SQLite database holds the records. Removing one does not automatically remove the other.

6. What happens if code or records are removed?

If someone deletes a source-code file

- **Before recompilation:** a currently running Java process will generally keep running with the classes already loaded in memory. The deleted file affects the next build/start, not the database file.
- **At the next build:** removing a required class such as `App.java`, `RecyclingHttpServer.java`, or `RecyclingDatabase.java` causes compilation or startup to fail because the remaining classes depend on it.
- **Removing a UI section or JavaScript handler:** the relevant page/button/refresh may disappear or stop working. If the matching server route is removed too, that API operation returns "Not found." If only one side is removed, the UI and backend no longer agree.
- **Removing styles only:** functionality may remain, but the page loses its intended appearance and responsive layout.
- **Removing comments/documentation:** normally does not change runtime behavior, but makes maintenance and onboarding harder.
- **Removing the empty Loginframe/MainFrame files:** based on the inspected project, this should not affect the current app because neither supplies the active browser UI or an entry point.

If someone deletes a supplier using the app

The supplier is removed only if it has no transaction history. The data layer checks this rule even if someone bypasses the browser button. If there is history, deletion is rejected and the records remain. Renaming is the supported way to correct a name while keeping history.

If someone deletes the database file

This is different from deleting Java code. The database contains saved records. If the database file is permanently removed, those records are gone; on the next launch, the app creates a new, empty database schema. Recovery then depends on a backup. Removing `target/` is different: that is generated compiled output and can be rebuilt.

Safe way to change or remove code

1. Identify the behavior the change is meant to remove; search for the page control, JavaScript action, API route, database method, and related tests/docs.
2. Back up the SQLite database and confirm the backup can be found; do not treat source deletion as data cleanup.
3. Change the UI and matching server/database surfaces together where appropriate. Preserve record-protection rules.
4. Compile with the existing launcher/build and run a focused manual or automated check for the affected flow.
5. Check both a successful path and expected failure paths (e.g., duplicate supplier, invalid amount, delete supplier with history).
6. Only remove files after confirming nothing still imports, calls, packages, or documents them.

7. How to explain it to colleagues

A short version you can say out loud

> "DAN KK Recycling System is a small local web app for the recycling centre. We open it in a browser, record each delivery's supplier, material, weight and rate, and it saves that information in a SQLite database on this computer. The dashboard shows centre totals and recent intake, while the Suppliers page tracks each partner's contribution. Java runs the local server; the browser is the user interface. The data file is separate from the code, so deleting code can break features, but deleting the database can erase our records. We keep historical supplier records protected from accidental deletion."

If someone asks, "How does one record get saved?"

"A staff member fills in the dashboard form. The browser sends the information to the Java server on this computer. The server checks it, calculates the value, writes it into SQLite, and replies that it was saved. The browser then requests fresh totals and redraws the dashboard."

If someone asks, "What should I be careful about?"

"Keep the launcher window open while using the app, and back up the database before code changes. A supplier with saved intakes cannot be deleted because its history matters. The app is designed for local use; it does not currently have logins, cloud sync, or automatic backup."

Simple analogy

The browser is the reception desk and forms. The Java server is the clerk who checks requests and coordinates work. The database is the filing cabinet. A change to the desk can make it harder to submit a record; a change to the clerk can break the rules; deleting the filing cabinet can lose records. Replacing the clerk's instructions (source code) does not by itself empty the filing cabinet (database).

8. Glossary and first checks when it does not work

Term	Plain meaning
Java	The programming language/runtime that runs the server and app logic.
HTTP server	The part that serves the page and receives browser requests.
API / route	A defined URL and action the browser uses to ask the server to do something.
SQLite	A small database stored as a file on this computer.
JDBC driver	The library that lets Java communicate with SQLite.
Source code	Human-readable project files from which the program is compiled.
Compiled class / JAR	Generated Java program files; normally rebuildable from source.
Loopback address	`127.0.0.1`, meaning “this same computer.”

Basic troubleshooting

- **Browser cannot connect:** check that the launcher window is still open; confirm the address is exactly `http://127.0.0.1:8765/`.
- **Launcher says Java is missing/wrong:** install/use JDK 25; the script checks both `java` and `javac`.
- **SQLite JAR is not cached:** follow the README instruction to connect once and run `mvn dependency:go-offline`.
- **A record is rejected:** check that supplier name is present and at most 160 characters, the material is from the list, weight/rate are positive, and the decimal places meet the limits.
- **Unexpected database error:** read the launcher console for the error; avoid deleting or replacing the database as a first troubleshooting step.

Scope note: this guide describes the source files present in the inspected workspace at the time it was prepared. The active source files are `App.java`, `RecyclingHttpServer.java`, and `RecyclingDatabase.java`; `MainFrame.java` is empty and `Loginframe.java` is an empty interface. A future code change may alter these details.